

Glaukopolis: Using Instruction Finetuning to Create a Small (32B) CTI LLM

Peter J. Worth Jr.,^{1,*} Ionut Cardei¹ and Md Tanvirul Alam¹

¹Department of Electrical Engineering and Computer Science, Florida Atlantic University, 777 Glades Road, 33431, Boca Raton, FL, United States

*Corresponding author. pworth@athenasecuritygroup.com

Abstract

Large language models (LLMs) are increasingly applied to cyber security tasks ranging from vulnerability analysis to cyber threat intelligence (CTI) reporting. However, generic foundation models often lack the domain knowledge, calibration, and reasoning needed for high-stakes CTI workflows, while frontier models raise cost, latency, and privacy concerns. Recent work from IBM on CyberPal.AI and its follow-on line “Toward Cybersecurity-Expert Small Language Models” demonstrates that expert-curated instruction fine-tuning (IFT) can transform small language models (SLMs) into capable cyber security assistants when paired with high-quality, domain-specific instruction datasets such as SecKnowledge and SecKnowledge 2.0 Levi et al. [2024, 2025]. In parallel, the CTI community has developed rich knowledge bases (MITRE ATT&CK, CVE, CAPEC, CWE, ENGAGE) and domain-specific benchmarks such as CTIBench and, most recently, AthenaBench—a dynamic, multi-task CTI benchmark that integrates live data sources and emphasizes applied security reasoning Strom et al. [2018], mit [b,c,a], Alam et al. [2024, 2025]. This paper surveys the research foundations underlying an ongoing effort at Florida Atlantic University and Athena Labs (Athena Security Group) to build instruction-tuned cyber security SLMs using a template-based IFT data generation system grounded in a CTI knowledge graph. We (1) review instruction fine-tuning and SLMs with emphasis on CyberPal and SecKnowledge; (2) describe the role of CTI knowledge bases and knowledge graphs in structuring cyber reasoning tasks; (3) situate Athena’s template-based IFT generator within this landscape; and (4) analyze the state of cyber security LLM benchmarking, with particular focus on AthenaBench as an emerging standard for CTI evaluation. We conclude by outlining research directions for aligning template-generated IFT corpora, cyber security SLMs, and dynamic CTI benchmarks.

Keywords instruction fine-tuning; small language models; cyber threat intelligence; CTI knowledge graphs; CyberPal; AthenaBench; reinforcement learning with verifiable rewards

Abbreviations ATT&CK, CAPEC, CoT, CTI, CVE, CWE, IFT, KEV, KG, LLM, NVD, RLHF, SFT, SLM, SOC, TTP

Introduction

LLMs have rapidly become integral to many security workflows: summarizing threat reports, mapping indicators to MITRE ATT&CK, triaging vulnerabilities, and assisting with detection engineering. Surveys of LLMs for security show use cases spanning vulnerability detection, malware analysis, CTI, phishing detection, and SOC automation, but also highlight persistent issues: hallucinations, shallow pattern matching rather than causal reasoning, and limited ability to operate on up-to-date threat information Levi et al. [2024], Liu et al. [2024].

Two trends have emerged in response:

- **Domain-specific models and datasets.** Models such as SecureBERT and CySecBERT pretrain or adapt transformers on cyber security corpora, improving representation quality for CTI and related tasks. Instruction-tuned cyber security assistants such as Lily-Cybersecurity-7B and IBM’s CyberPal models further specialize LLMs for interactive security workflows using cyber-focused instruction–response pairs Levi et al. [2024].

- **Domain-specific benchmarks.** Benchmarks such as CyberMetric, CyberBench, SecEval, CTIBench, and SEvenLLM-Bench evaluate security knowledge, reasoning, and incident analysis in LLMs Alam et al. [2024], Ji et al. [2024], Liu et al. [2024].

However, many existing benchmarks are static and quickly fall out of sync with the evolving threat landscape. AthenaBench addresses this by introducing a *dynamic CTI benchmark* that continuously draws from live sources (MITRE ATT&CK, NVD, current APT reports) to construct tasks targeting CTI knowledge, vulnerability severity, root cause mapping, threat actor attribution, mitigation strategies, and attack technique extraction Alam et al. [2025].

In parallel, the IFT design document by Cardei describes an *IFT Dataset Generation System* that compiles high-level templates into (instruction, question, answer) triples grounded in an ASG Neo4j CTI graph built from MITRE and other CTI sources Cardei [2025]. This system is intended to produce large, structured, and incrementally refreshable cyber security IFT corpora for training SLMs specialized to CTI workflows.

This paper positions that work in its research context:

- Section 3 reviews instruction fine-tuning and small language models, emphasizing IBM’s CyberPal line.
- Section 4 discusses CTI, CTI knowledge bases, and Ariadne—the Athena CTI knowledge-graph substrate that supplies every fact in the downstream training corpus—together with the template language that compiles graph queries into IFT triples.
- Section 5 surveys cyber security LLM benchmarks, highlighting AthenaBench and its relationship to CTIBench and other efforts.
- Section 6 describes Glaukopis itself: the template-based SFT model for the Athena-CTI-LLM family, including the chained, multi-stage training recipe and the evaluation suites against which it is measured.

Chaining SFT: Multi-Stage Supervised Fine-Tuning

A recurring design choice in modern post-training is to compose several supervised fine-tuning (SFT) stages into a chain rather than to perform a single, monolithic instruction tune. In a chained SFT pipeline, the base pretrained model is adapted to its target deployment through an ordered sequence of stages, each with its own dataset, optimization hyperparameters, and intended outcome; the output checkpoint of each stage becomes the initialization for the next. Capabilities are thereby accumulated compositionally instead of competing for capacity within a single fine-tune, and stages can be re-run independently when their underlying data changes.

The staged structure of modern LLM post-training is by now well established. InstructGPT formalized a three-phase pipeline—supervised instruction fine-tuning, preference modeling, and reinforcement learning from human feedback—in which each phase has its own data and objective and the order of the stages is load-bearing for the final model’s behavior Ouyang et al. [2022]. FLAN and FLAN-T5 showed earlier that instruction tuning is most effective when it follows a well-pretrained base rather than replacing pretraining, that the diversity and structure of instruction data matter at least as much as parameter count, and that the gains of instruction tuning persist across model scales Wei et al. [2022a], Chung et al. [2022]. The domain-adaptive pretraining literature, notably the “Don’t Stop Pretraining” line, demonstrated that continued training of a general model on in-domain text yields measurable downstream improvements and that the order and composition of training stages materially affect final task performance Gururangan et al. [2020].

Several practical motivations recur across these results. First, separating broad domain absorption from precise task behavior reduces destructive interference: training jointly on noisy domain text and structured task triples tends to either underfit the structured task or overfit narrow templates, while staging gives each objective its own learning-rate, schedule, and packing regime Gururangan et al. [2020], Ouyang et al. [2022]. Second, reasoning supervision is more effective when it is layered on top of a model that already commands the underlying domain vocabulary and task formats than when it is mixed in from the start; this is the design that motivated the chain-of-thought enrichment stages in CyberPal/SecKnowledge 2.0 and analogous reasoning-focused fine-tunes Wei et al. [2022b], Levi et al. [2025]. Third, staging makes pipelines incrementally refreshable: when only one data source updates, only the stages that consume it need to be re-run, and intermediate checkpoints can be evaluated to attribute downstream gains to specific stages.

In the cyber security setting in particular, the CyberPal.AI / SecKnowledge line operationalizes this pattern explicitly, separating expert instruction-schema construction, knowledge-grounded answer synthesis, and chain-of-thought rationale enrichment into distinct dataset-construction and training stages, with measurable benefits from the staged structure Levi et al. [2024, 2025]. Glaukopis, described in Section 6, adopts the same general philosophy with a four-stage chain tailored to template-generated CTI instruction data and the AthenaBench / CyberSOCEval / CyberMetric evaluation portfolio.

Instruction Fine-Tuning: History and Context

Instruction fine-tuning (IFT) has emerged as a foundational paradigm for aligning large language models (LLMs) with human intent and task-oriented usage. Rather than treating language modeling as an end in itself, IFT reframes downstream interaction as the problem of mapping natural-language instructions to correct, task-specific outputs. Although early work on task fine-tuning existed prior to large-scale transformers, the modern formulation of instruction fine-tuning coalesced through three seminal research efforts between 2021 and 2022. Together, these works established the empirical phenomenon, the alignment pipeline, and the scalability properties that underpin nearly all contemporary instruction-tuned models, including domain-specialized systems such as CyberPal.AI and the Athena CTI instruction framework.

FLAN: Instruction Tuning as a Generalization Paradigm

The first widely recognized formulation of instruction fine-tuning was introduced by Wei et al. in *Finetuned Language Models Are Zero-Shot Learners* (FLAN) Wei et al. [2022a]. This work demonstrated that large pretrained language models, when fine-tuned on a sufficiently diverse collection of tasks expressed uniformly as natural-language instructions, exhibit strong zero-shot generalization to unseen tasks.

FLAN unified dozens of existing NLP datasets by converting each task into multiple instruction templates (e.g., question answering, sentiment analysis, textual entailment, translation). The authors showed that instruction formatting itself—explicitly describing the task in natural language—was a critical factor in enabling generalization, often rivaling or surpassing few-shot prompting on much larger models.

The central insight of FLAN is that *instruction-following is a learnable capability* rather than an emergent property of scale alone. By training on heterogeneous instruction–response pairs, the model internalizes a latent task distribution that allows it to interpret novel instructions without task-specific fine-tuning. This result established instruction fine-tuning as a distinct and powerful paradigm, forming the conceptual foundation for subsequent work in alignment and domain adaptation.

InstructGPT: Instruction Following and Alignment with Human Feedback

While FLAN established the effectiveness of instruction fine-tuning for generalization, it did not directly address issues of safety, helpfulness, or alignment with human preferences. These concerns were tackled by Ouyang et al. in *Training Language Models to Follow Instructions with Human Feedback* (InstructGPT) Ouyang et al. [2022].

InstructGPT introduced a three-stage training pipeline that remains the basis for many production LLMs:

1. **Supervised instruction fine-tuning (SFT)** on human-written instruction–response pairs;
2. **Preference modeling** using human rankings of multiple model outputs;
3. **Reinforcement learning from human feedback (RLHF)** to optimize model behavior toward preferred responses.

A key result of this work was that relatively small models, when instruction-tuned and aligned via RLHF, were preferred by human evaluators over much larger pretrained models. This demonstrated that alignment and instruction-following quality can outweigh raw model scale.

From a historical perspective, InstructGPT complements FLAN by transforming instruction fine-tuning from a purely performance-driven technique into a broader alignment framework. Together, FLAN and InstructGPT established instruction fine-tuning as both a generalization mechanism and a control interface between humans and large language models.

FLAN-T5: Scaling Instruction Fine-Tuning

The third pillar of instruction fine-tuning was established by Chung et al. in *Scaling Instruction-Finetuned Language Models* (FLAN-T5) Chung et al. [2022]. This work systematically explored how instruction fine-tuning behaves across model sizes, task mixtures, and dataset scales.

FLAN-T5 expanded the instruction corpus to thousands of tasks and demonstrated that instruction fine-tuning consistently improves performance across a wide range of downstream benchmarks, including reasoning and knowledge-intensive tasks. Importantly, the authors showed that instruction fine-tuning benefits models of all sizes, from small to very large, and that gains persist even as base models scale.

This result established instruction fine-tuning as a *scalable and robust training paradigm*, rather than a technique limited to a narrow regime. It also reinforced the idea that the diversity and structure of instruction data are as important as parameter count—a conclusion that directly motivates modern domain-specialized instruction datasets.

Implications for Domain-Specialized Instruction Tuning

Together, FLAN, InstructGPT, and FLAN-T5 define the historical and conceptual foundations of instruction fine-tuning:

- FLAN establishes that instruction-following can be learned and generalized;
- InstructGPT integrates instruction fine-tuning with human alignment objectives;
- FLAN-T5 demonstrates that instruction fine-tuning scales across model sizes and task mixtures.

Recent domain-specific efforts—such as CyberPal.AI and SecKnowledge Levi et al. [2024, 2025]—extend these principles by grounding instruction data in structured cyber security knowledge bases. The Athena CTI instruction framework further advances this trajectory by combining symbolic, graph-based instruction templates with benchmark-driven iteration via AthenaBench. In this sense, modern

CTI-focused instruction fine-tuning can be viewed as a third-generation evolution of the original FLAN paradigm: instruction learning grounded not only in natural language, but in explicit domain structure and formal knowledge representations.

Instruction Fine-Tuning and Small Language Models From Pretraining to Instruction-Following

Pretrained LLMs are optimized for next-token prediction on broad text corpora, not for following explicit user instructions. Instruction tuning—also called instruction fine-tuning (IFT)—fine-tunes such models on labeled datasets of instructions and target outputs, improving their ability to interpret prompts as tasks rather than mere text prefixes Wei et al. [2022a], Ouyang et al. [2022].

Key characteristics of instruction tuning include:

- **Data format.** Each sample consists of an instruction, optional context, and a desired output.
- **Supervised objective.** The model is trained to maximize log-likelihood of the reference output given the instruction and context, often combining a standard language modeling loss with an instruction-following loss.
- **Generalization across tasks.** Instruction datasets deliberately mix many task types (QA, classification, summarization, transformation) so that the fine-tuned model can generalize to novel instructions, as shown by InstructGPT and FLAN Wei et al. [2022a], Ouyang et al. [2022].

IFT is now routine in the model lifecycle: base models (e.g., LLaMA-like, Granite, Qwen) are first pretrained, then instruction-tuned, and often further refined via reinforcement learning from human feedback (RLHF) and safety fine-tunes Ouyang et al. [2022].

Instruction Fine-Tuning for Cyber Security

Cyber Security introduces domain-specific challenges:

- Technical jargon and heterogeneous schemas (CVE, CWE, CAPEC, ATT&CK).
- Need for structured, evidence-grounded outputs (e.g., CVSS vectors, CWE IDs, ATT&CK mappings).
- Rapid concept drift as new vulnerabilities and campaigns appear.

Several efforts address this via cyber-specific instruction data:

- **SEvenLLM.** Ji et al. construct a bilingual cyber security instruction dataset and a CTI benchmark (SEvenLLM-Bench) from real incident reports, converting raw text into supervised QA, extraction, and reasoning tasks across many task types Ji et al. [2024].
- **CyberBench / CyberInstruct.** Liu et al. create CyberInstruct, a set of cyber-specific instruction tasks drawn from logs, alerts, and vulnerability descriptions, used to train models evaluated on CyberBench Liu et al. [2024].
- **Lily-Cybersecurity-7B.** Lily is a Mistral-7B fine-tune on hand-crafted cyber security and hacking instructions, later enriched by an LLM for style and consistency Liu et al. [2024].

The **CyberPal.AI** line of work from IBM pushes this further by tightly integrating CTI knowledge bases and detection artifacts into an expert-driven instruction pipeline:

- **SecKnowledge (CyberPal 1.0).** Levi et al. build an instruction dataset by mining MITRE ATT&CK, Sigma rules, SIEM rules, adversary emulation plans, and threat reports, then using domain experts to define instruction schemas (e.g., mapping logs to ATT&CK techniques, suggesting mitigations) and populate them semi-automatically Levi et al. [2024].
- **IFT results.** CyberPal models instruction-tuned on SecKnowledge (based on LLaMA-like and other open models) outperform base models by up to roughly 24% on security tasks and show improved robustness on CTI benchmarks Levi et al. [2024].

The follow-on “**Toward Cybersecurity-Expert Small Language Models**” paper extends this to SecKnowledge 2.0, enriching instructions with chain-of-thought (CoT) rationales and designing a pipeline with expert-in-the-loop validation and LLM-guided knowledge retrieval, then training cyber security expert SLMs (4B–20B parameter models) that rival or surpass much larger general models on CTI tasks Levi et al. [2025].

These works demonstrate that expert-designed, knowledge-grounded instruction datasets can transform general LLMs into specialized CTI assistants with strong reasoning and reduced hallucination—precisely the niche targeted by the Athena template-based IFT system.

Small Language Models for Cyber Security

SLMs—roughly 2–20B parameters—are increasingly attractive for cyber security:

- **Deployment constraints.** Many SOC and CTI environments require on-premises, air-gapped, or “bring-your-own-model” setups for regulatory and confidentiality reasons.
- **Latency and cost.** Incident response and detection engineering often require low-latency responses and large volumes of queries; SLMs are simpler to host and scale.
- **Specialization.** With strong domain IFT, SLMs can match or exceed larger general models on specialized tasks while being easier to retrain as CTI sources evolve Levi et al. [2025].

CyberPal 2.0 explicitly targets this: it compares cyber security expert SLMs (4B–20B) against both larger open models and proprietary systems across CTI knowledge tasks, finding that well-instruction-tuned SLMs can achieve competitive or superior performance on several domains, especially when trained with CoT-enriched instructions and CTI-aware formatting Levi et al. [2025].

The emerging picture is that *IFT + SLM + domain knowledge* is a pragmatic recipe for CTI: the heavy lifting (linguistic knowledge) comes from pretraining; domain semantics and task formats come from IFT; runtime and deployment practicality come from SLM size. The Athena IFT dataset generation system is designed to feed this recipe by generating large quantities of high-fidelity instruction triples directly from CTI knowledge bases Cardei [2025], and the specific Glaukosis instantiation of this recipe—including the four-stage SFT chain, its per-stage datasets and hyperparameters, and its evaluation harness—is described in Section 6.

Cyber Threat Intelligence and Knowledge Graphs: Ariadne, the Athena CTI Substrate

CTI Concepts and Workflows

Cyber Threat Intelligence (CTI) is commonly defined as the collection, analysis, and dissemination of information about threats and adversaries to inform defensive decisions McMillan [2013]. CTI workflows include:

- Ingesting unstructured reports (vendor blogs, whitepapers, advisories).
- Structuring them into entities and relations: vulnerabilities (CVEs), weaknesses (CWEs), attack patterns (CAPEC, ATT&CK), threat actors, campaigns, malware, tools, TTPs.
- Reasoning over this structure to answer questions such as:
 - Which techniques does this malware implement?
 - Which mitigations cover these techniques on a given platform?
 - Which threat actor is most likely behind this cluster of activity?
 - How severe and exploitable is a new CVE in our environment?

CTI sharing and CTI knowledge graphs emphasize the importance of standardized schemas (e.g., STIX/TAXII), community knowledge bases (MITRE ATT&CK, CAPEC, CWE), and APIs like NVD for building structured representations that can be consumed by automation Strom et al. [2018], mit [b,c,a].

MITRE Ecosystem and Related Schemas

The MITRE ecosystem underpins much of modern CTI:

- **MITRE ATT&CK.** A globally accessible knowledge base of adversary tactics, techniques, and sub-techniques, originally released in 2013 and formally documented in “MITRE ATT&CK: Design and Philosophy” Strom et al. [2018]. ATT&CK represents adversary behavior as a graph where attack-pattern nodes (techniques) relate to software, groups (threat actors), data sources, and mitigations.
- **CVE and NVD.** The Common Vulnerabilities and Exposures system and NIST’s National Vulnerability Database provide machine-readable vulnerability records with descriptions, affected products, CVSS metrics, and links to related CWE and CAPEC entries mit [b].
- **CWE.** Common Weakness Enumeration captures abstract classes of software/hardware weaknesses, often linked from CVEs and CAPEC attack patterns mit [c].
- **CAPEC.** A catalog of attack patterns describing how adversaries exploit weaknesses and how those attacks can be mitigated, with explicit links to CWE and often implicitly to ATT&CK techniques mit [a].
- **MITRE ENGAGE.** A framework for adversary engagement strategies, which can be integrated into CTI graphs as a defensive complement to ATT&CK Cardei [2025].
- **MITRE D3FEND.** A complementary knowledge graph of defensive countermeasures structured to mirror the offensive ATT&CK ontology, providing a controlled vocabulary for the defensive techniques that map back to specific attack techniques.

Ariadne: The Athena CTI Knowledge Graph

Ariadne is the codename for the Athena CTI knowledge-graph substrate—the populated Neo4j database that all downstream Athena components (template generation, SFT, evaluation harness) consume as their single upstream source of CTI facts. The design premise is simple: every fact that can appear in an Athena SFT example must first exist as a node or edge in Ariadne. The graph is therefore both the training corpus’s data layer and the evaluation harness’s reference oracle, and the rigor of Athena’s downstream guarantees rests on Ariadne’s source coverage and refresh discipline.

Source coverage.

Ariadne ingests twelve public CTI sources spanning offensive technique catalogues, vulnerability and weakness databases, exploitation evidence, defensive countermeasures, and detection content:

- **MITRE ATT&CK** (Enterprise, STIX 2.1 JSON) and **MITRE ENGAGE** (JSON) for offensive and adversary-engagement technique catalogues.
- **MITRE CAPEC** (XML) and **MITRE CWE** (XML) for attack patterns and weakness classes.
- **MITRE D3FEND** (JSON-LD + SPARQL JSON, pinned to v1.4.0 for substrate stability) for defensive countermeasures aligned to ATT&CK.
- **CVE Project (CVE 5.0)** for the canonical vulnerability records.
- **NVD CPE/CVSS feeds** (per-year gzipped NDJSON) for the vendor-product affected-component and severity overlays.
- **CISA KEV** (Known Exploited Vulnerabilities catalogue, JSON) for the authoritative “actively exploited” signal.
- **FIRST EPSS** (gzipped CSV) for the daily exploitation-probability scores keyed to CVE identifiers.
- **Sigma rules** (YAML) for community detection content keyed to ATT&CK techniques.
- **ExploitDB** (CSV) and **PoC-in-GitHub** (JSON year folders) for the weaponized-exploit evidence layer associated with specific CVEs.

ATT&CK, ENGAGE, CAPEC, CWE, CISA KEV, D3FEND, and Sigma are loaded in their entirety because the catalogues themselves are bounded. CVE, NVD, EPSS, ExploitDB, and PoC-in-GitHub are scoped to 2024 onwards. The cutoff is enforced both at clone time (sparse-clone path selection) and at parse time (per-source date filters), and is the principal mechanism by which Ariadne keeps the populated graph at a working size of approximately 280 000 nodes and 480 000 edges—large enough to support multi-hop, cross-schema reasoning over the modern threat surface, small enough to fit on a single Neo4j instance with practical query latency.

Refresh discipline.

The populator is idempotent: every Cypher write uses **MERGE** against per-node uniqueness constraints (**stix.id** for STIX-bearing nodes plus natural-key constraints on identifiers such as CVE IDs, CWE IDs, and D3FEND IDs), so re-running the populator never produces duplicates. The download layer applies three distinct refresh policies. *Always-refresh* sources (FIRST EPSS, NVD current-year feed) are re-downloaded on every populator run, because both update daily; EPSS specifically pulls the prior day’s snapshot and overwrites

the local CSV in place. *Skip-if-cached HTTPS* sources (CAPEC, CWE, CISA KEV, D3FEND) are downloaded once and re-used on subsequent runs unless the cached file is explicitly deleted; D3FEND additionally pins a version constant so substrate stability is byte-exact across runs until the version is bumped. *Skip-if-cached Git* sources (ATT&CK, ENGAGE, CVE, Sigma, ExploitDB, PoC-in-GitHub) are sparse-cloned once and refreshed by manual **git pull** or by deleting the source directory. The pattern that emerges in production is a nightly cron run that picks up the daily EPSS and current-year NVD churn while leaving the static catalogues untouched until their cache is manually cleared.

Graph construction order.

Ariadne is built in a fixed dependency order chosen to ensure that cross-framework edges always have both endpoints present when the relationship is created. Framework nodes are populated first—ATT&CK, CAPEC, CWE, ENGAGE, D3FEND, and Sigma—because these are the bounded catalogues that define the controlled vocabulary the rest of the graph references. The vulnerability surface follows: CVE records, NVD CPE/CVSS overlays, CISA KEV exploited-vulnerability flags, and FIRST EPSS probability scores. The weaponized-exploit layer (ExploitDB rows, PoC-in-GitHub entries) is last, since its rows reference CVE nodes that must already exist. Intra- and cross-framework relationships are then materialized, including the canonical CVE→CWE root-cause edges, CWE→CAPEC and CAPEC→ATT&CK pivots, ATT&CK→Mitigation and ATT&CK→Data-Component links, and the D3FEND↔ATT&CK defensive overlay.

Downstream consumption.

Ariadne is the upstream substrate for the template generation module described in Section 4.4, which traverses the graph to produce the IFT triples consumed by the SFT chain (Section 6.4). The same Neo4j connection parameters used by the populator are read by the template generator, ensuring that the database name (**athena-cti-db**) and authentication credentials are the single source of truth across the entire Athena pipeline. The fact that every SFT training row is materialized from a Cypher query against Ariadne is what makes the chained-SFT pipeline auditable in the sense discussed in Section 6.1: every answer in every training row is traceable to specific node and edge identifiers in the populated graph, which in turn are traceable to specific public-source rows by their natural keys.

The Athena Template Language

The *Instruction Fine Tuning Dataset Design* document by Cardei defines an IFT Dataset Generation System built around the Ariadne graph and a rich template language Cardei [2025]. Key features include:

- **CTI source support.** The template generator binds against the same twelve sources ingested by Ariadne (Section 4.3), grouped for template-authoring purposes into three categories: (1) structural/graph sources (MITRE ATT&CK, CVE, CAPEC, CWE, MITRE ENGAGE, EPSS, CISA KEV, D3FEND), bound directly via Cypher; (2) LLM-assisted sources (SIGMA rules, MITRE CAR, Wazuh rules), where natural-language fields are augmented by LLM extraction before being merged into the graph; and (3) proprietary sources (e.g., Flashpoint Ignite) where licensed feeds are normalized into the same node and edge schema.

- **IFT triple format.** Each generated data point is an (instruction, question, answer) triple: the instruction specifies the role and style (e.g., “You are a CTI expert that gives precise and concise answers”), the question is the user-facing CTI query, and the answer is the ground-truth response derived from the CTI graph.
- **Template language.** Templates provide a high-level, declarative way to bind CTI graph nodes and produce triples. They support aliases for CTI types; attribute access through nested properties; filters (using equality and inequality) applied to list properties with any/all semantics; local variables bound to nodes; random selection from lists; list-comprehension-like constructs to expand relationships into lists in the output; invisible sections that define bindings and constraints without appearing in text; and source disambiguation via a `source#type` prefix (e.g., `attck#attack-pattern` versus `capec#attack-pattern`) Cardei [2025].
- **Positive and negative examples.** The template language explicitly supports negative constraints (e.g., mitigations that do *not* mitigate this technique or data components that do *not* detect a technique), enabling the generation of hard negatives that teach models what does *not* follow from the CTI graph.
- **Cross-source reasoning templates.** Example templates mimic complex reasoning chains, such as: given a CVE description, identify the corresponding CWE weakness, then find a CAPEC attack pattern with high likelihood of attack using that weakness, then find an ATT&CK technique implementing that attack pattern.
- **Incremental generation.** The generator allows version and date filters at both document and concept level. Users can specify version ranges and last-modified intervals per CTI source; relationships are only used if at least one endpoint and the relationship fall within those windows.
- **Generation control and reporting.** A JSON job configuration file specifies CTI version ranges, template texts, per-template count limits and coverage fractions, output directories, and dataset versioning. The generation API produces both per-template triple files and a global report summarizing counts, coverage, and errors, which can be post-processed into human-readable formats Cardei [2025].

Conceptually, this system is a symbolic counterpart to the LLM-assisted instruction pipelines used in CyberPal and SEvenLLM: where CyberPal uses LLMs and experts to synthesize security instructions from unstructured sources, the Athena system uses a declarative template language over a CTI graph to generate strongly grounded and precisely controllable instruction triples.

CTI Knowledge Graphs and LLMs

Recent work shows that LLMs and CTI knowledge graphs are synergistic. Various efforts use LLMs to extract triples from CTI text and populate knowledge graphs, then perform link prediction and reasoning over those graphs to produce actionable recommendations Fieblinger et al. [2024], Zhang et al. [2023], Wu et al. [2022], Yu et al. [2024], Zhao et al. [2024]. Other systems build CTI-KGs directly from ATT&CK, CVE, threat reports, and open sources, then query them using LLMs or hybrid systems for tasks such as threat hunting and reporting.

The Ariadne / Athena IFT design follows a complementary pattern: *knowledge graph* $\rightarrow$ *templates* $\rightarrow$ *IFT triples* $\rightarrow$ *SLM*,

with AthenaBench serving as a *knowledge graph* $\rightarrow$ *evaluation tasks* pipeline. This alignment is central to the next section.

Cyber Security LLM Benchmarks and AthenaBench

Why Cyber-Security-Specific Benchmarks?

General benchmarks such as GLUE, MMLU, and BIG-Bench measure broad language understanding and world knowledge, and certain security-adjacent capabilities (e.g., information-security multiple-choice questions in MMLU) appear as minor subsets, but these suites are not designed to probe the workflows that actually define security practice. They lack the structured schemas, evidence-grounded reasoning, and threat-current content that distinguish cyber security tasks from general knowledge work. The gaps that motivate domain-specific benchmarking are concrete:

- **Schema fidelity.** Security workflows operate over standardized identifiers and structured artifacts (CVE, CWE, CAPEC, CVSS vectors, MITRE ATT&CK technique IDs, STIX/TAXII objects). General benchmarks do not test whether a model produces well-formed outputs in these schemas, which is the difference between a usable assistant and one that requires constant human re-formatting.
- **Applied reasoning.** CTI and SOC analysts rarely need pure recall; they need chained inferences (e.g., from a CVE description to the underlying weakness class, then to relevant attack patterns and detection opportunities), severity estimation under partial information, and abductive attribution from observed TTPs. These reasoning shapes are largely absent from general benchmarks.
- **Temporal relevance.** The threat landscape evolves on a weekly cadence (new CVEs, new APT activity, new exploitation campaigns), so static benchmarks rapidly drift from operational reality and become measures of stale memorization rather than current competence Alam et al. [2025].
- **Data contamination.** Many cyber security sources (MITRE ATT&CK, the NVD, public threat reports) are heavily represented in LLM pretraining corpora. A static benchmark drawn from those same sources risks measuring recall of training data rather than genuine reasoning, an effect that dynamic benchmarks aim to mitigate Alam et al. [2025].

In response, the past two years have seen a rapid expansion of cyber-security-specific benchmarks, which can be grouped roughly into four overlapping families: *knowledge benchmarks*, focused on factual recall of security concepts and terminology; *secure-coding and offensive-capability benchmarks*, focused on whether models help or hinder secure software development and adversarial use; *CTI and SOC benchmarks*, focused on applied threat-intelligence and operations reasoning; and *dynamic / live-source benchmarks*, focused on resistance to data staleness and contamination. Important entries include:

- **CyberMetric.** A retrieval-augmented, certification-style multiple-choice benchmark by Tihanyi et al. covering nine cyber security domains, released in graduated sizes (80, 500, 2000, and 10000 questions) to support both rapid evaluation and statistically robust comparison Tihanyi et al. [2024].

- **SecEval.** A multiple-choice benchmark of cyber security knowledge questions spanning multiple security subdomains, used to evaluate factual coverage of foundation models on standard certification-style content Hu et al. [2024].
- **CyberBench.** A multi-task benchmark for cyber security LLMs that includes classification, question answering, and log/report analysis, accompanied by the CyberInstruct training corpus used by the same authors to build cyber-tuned models Liu et al. [2024].
- **CTIBench.** A dedicated CTI benchmark covering CTI knowledge, root-cause mapping, vulnerability severity prediction, threat-actor attribution, threat-knowledge graph construction, and attack-technique extraction; constructed from curated CTI corpora and designed for reasoning-intensive evaluation Alam et al. [2024].
- **SEvenLLM-Bench.** A bilingual CTI benchmark derived from real incident reports, designed alongside the SEvenLLM-Instruct dataset and emphasizing cross-lingual security comprehension Ji et al. [2024].
- **Purple Llama CyberSecEval and CyberSecEval 2.** Benchmarks from Meta evaluating secure-code generation, susceptibility to assisting with cyberattacks, prompt-injection robustness, and related risk dimensions Bhatt et al. [2023, 2024].
- **WMDP.** The Weapons of Mass Destruction Proxy benchmark by Li et al. includes a cyber security subset measuring potentially hazardous cyber-offense knowledge and is widely used to study unlearning and safety-capability tradeoffs in security-adjacent capabilities Li et al. [2024].
- **AthenaBench.** A dynamic CTI benchmark that draws from live sources (MITRE ATT&CK, NVD, CISA advisories, threat reports) and explicitly targets the staleness and contamination problems that limit static suites; discussed in detail below Alam et al. [2025].

Together, these benchmarks shift evaluation from generic question answering toward security-aware reasoning, but they cover very different surfaces—factual recall, secure-code generation, applied CTI reasoning, and dynamic resistance to drift—and no single benchmark is sufficient on its own. Glaukosis is therefore evaluated against a portfolio of complementary suites, with particular weight on CTIBench and AthenaBench (which directly target CTI reasoning), CyberMetric at the 2000- and 10 000-question tiers (which provides statistically robust factual coverage), and CyberSOCEval’s malware and threat-intelligence test suites (which probe operationally relevant SOC reasoning). The remainder of this section describes these benchmarks in more detail.

CyberMetric

CyberMetric is a multiple-choice cyber security knowledge benchmark constructed by Tihanyi et al. using a retrieval-augmented generation (RAG) pipeline over an authoritative corpus of cyber security standards, frameworks, NIST publications, and academic and practitioner references Tihanyi et al. [2024]. Candidate questions are generated by an LLM conditioned on retrieved source material, then filtered and validated by human cyber security experts before inclusion in the released dataset. The result is a benchmark whose questions are grounded in citable source material rather than constructed from scratch, which mitigates the common problem of LLM-as-author benchmarks whose answer keys reflect the generating model’s biases more than the underlying domain.

CyberMetric covers nine cyber security domains: network security, cryptography, identity and access management, security policies and governance, threats and vulnerabilities, security operations, application and system security, cloud security, and incident response. It is released in four cumulative sizes—80, 500, 2 000, and 10 000 questions—designed to support different evaluation regimes: the 80- and 500-question tiers for rapid iteration and ablation, and the 2 000- and 10 000-question tiers for statistically robust comparison between models where small accuracy differences must be distinguished from sampling noise Tihanyi et al. [2024].

In Glaukosis, CyberMetric is used at the 2 000- and 10 000-question tiers. These tiers were chosen for three reasons. First, sample size: at 10 000 questions the standard error on accuracy estimates is approximately 0.5 percentage points, which is fine-grained enough to detect the kind of single-digit improvements typically produced by an additional SFT stage in the chained recipe (Section 6.4). Second, domain coverage: the larger tiers include enough questions per domain to support per-domain breakdowns, which lets us identify whether a given training-stage change improves balanced cyber security knowledge or only narrow subdomains. Third, comparability: published evaluations of contemporary models—both general-purpose frontier models and cyber-specialized SLMs—report CyberMetric scores at the 2 000- and 10 000-question tiers, allowing direct comparison without re-running baselines.

CyberMetric’s principal limitation, shared with all multiple-choice benchmarks, is that it measures recognition rather than generation: a model can score well on CyberMetric without being able to produce structured CTI outputs or coherent multi-step reasoning. For this reason it is used in Glaukosis as one component of a broader evaluation portfolio rather than as a primary metric, and is paired with reasoning-oriented suites (CTIBench, AthenaBench) and operationally focused suites (CyberSOCEval) that exercise generation and applied reasoning.

CyberSOCEval

CyberSOCEval is an evaluation suite oriented around security-operations-center (SOC) tasks, providing test suites that target operationally meaningful reasoning rather than pure knowledge recall Meta AI / Purple Llama Team [2025]. Whereas CyberMetric measures what a model knows about cyber security as a body of knowledge, and CTIBench/AthenaBench measure what a model can reason over a CTI knowledge graph, CyberSOCEval measures whether a model can perform the kind of analytic work that occupies an analyst inside a SOC: classifying observed artifacts, correlating evidence, and producing actionable summaries.

In Glaukosis, CyberSOCEval is used along two of its test suites:

- **Malware analysis suite.** Tasks in this suite present malware-related artifacts (samples, behavioral indicators, or analyst-style descriptions) and probe whether the model can extract family/variant attribution, characterize behavior in a structured way (e.g., what techniques the artifact exhibits), and identify defensive implications. This suite is operationally important because malware-related reasoning is one of the most frequent generation tasks asked of CTI assistants and is also one of the most failure-prone: errors here typically take the form of confident misattribution rather than refusal, which is more harmful than abstention.

- **Threat-intelligence suite.** Tasks here exercise the analyst-facing CTI reasoning chain: ingesting raw report or alert content, extracting TTPs and indicators, mapping them to standardized schemas (ATT&CK techniques, mitigations, threat-actor designations where supported by evidence), and producing structured CTI summaries. This complements AthenaBench's task structure but emphasizes free-form narrative input over knowledge-graph-derived queries, which makes it a useful test of how well a CTI SLM generalizes from training data structured by the Athena template generator to less-structured real-world inputs.

These two suites are particularly relevant to Glaukopis because they exercise capabilities that the chained-SFT pipeline (Section 6.4) is explicitly designed to develop. Stage 1's Phase B AthenaBench-catalog drill and Stage 2's TAA narrow drill train the model on schema-faithful answers to graph-derived questions, and Stage 3 (the CyberSOCEval letter-set drill) is itself a targeted preparation against this benchmark family; the malware and threat-intelligence suites then test whether the capabilities cultivated through these stages transfer to operationally realistic narrative inputs. Their inclusion alongside CyberMetric (factual coverage), CTIBench (CTI reasoning), and AthenaBench (dynamic CTI reasoning) gives the evaluation portfolio coverage across the four benchmark families identified above: knowledge, secure-coding/offensive capability via Purple Llama suites as needed, applied CTI reasoning, and operational SOC reasoning Tihanyi et al. [2024], Alam et al. [2024, 2025], Bhatt et al. [2023, 2024], Meta AI / Purple Llama Team [2025].

AthenaBench: A Dynamic CTI Benchmark

AthenaBench is introduced as a dynamic benchmark suite for evaluating LLMs on realistic, knowledge-intensive CTI tasks and explicitly extends CTIBench Alam et al. [2025, 2024]. Its key innovations include:

- **Dynamic data construction.** Instead of static corpora, AthenaBench connects to live CTI data sources and APIs (MITRE ATT&CK, NVD, CISA advisories, threat reports) to continuously generate benchmark samples within configurable time windows. Tasks such as root cause mapping (CVE→CWE) and severity prediction (CVSS) are drawn from recent CVEs and weaknesses, mitigating training-set leakage and static knowledge effects Alam et al. [2025].
- **Six complementary tasks.** AthenaBench defines: CTI Knowledge Test (CKT), Root Cause Mapping (RCM), Vulnerability Severity Prediction (VSP), Threat Actor Attribution (TAA), Risk Mitigation Strategy (RMS), and Attack Technique Extraction (ATE). Together, they probe CTI knowledge, root cause reasoning, structured severity prediction, abductive attribution, defensive planning, and TTP extraction Alam et al. [2025].
- **Enhanced metrics and aggregated score.** AthenaBench improves task-specific metrics and defines a unified aggregated score, making it easier to compare LLMs' CTI reasoning capabilities along different dimensions Alam et al. [2025].
- **Model evaluation.** The authors evaluate 12 LLMs, including GPT-4, GPT-4o, GPT-5, Gemini-2.5 Flash/Pro, Qwen variants, Llama 3/3.1/3.3, and a Llama-Primus merged model. Proprietary models (GPT-5, Gemini-2.5 Pro) lead overall, but all models

struggle on reasoning-intensive tasks such as TAA and RMS, especially open-source ones Alam et al. [2025].

By using live CTI sources and emphasizing reasoning over current threats, AthenaBench moves CTI benchmarking closer to real-world analyst workloads and provides a stringent target for cyber security expert SLMs.

Relationship to Athena's IFT Dataset Generator

AthenaBench and the Athena IFT generator are complementary components of a full CTI LLM lifecycle:

- **Training side (IFT generator).** Uses CTI graph data (ATT&CK, CVE, CAPEC, CWE, ENGAGE, etc.) and a declarative template language to generate instruction triples that drill CTI knowledge, multi-hop reasoning, negative cases, and structured outputs Cardei [2025].
- **Evaluation side (AthenaBench).** Dynamically constructs CTI tasks from the same or similar underlying sources (ATT&CK, NVD, CISA, threat reports) but with different pipelines and sampling strategies, providing independent, evolving test sets to evaluate model generalization Alam et al. [2025].

Many of the template patterns in the design document directly mirror AthenaBench tasks:

- CVE→CWE reasoning templates align with RCM.
- Templates that traverse CVE→CWE→CAPEC→ATT&CK map onto knowledge and mitigation reasoning tested in CKT and RMS.
- Templates involving malware, techniques, platforms, and data components parallel the kind of “who uses what and how is it detected” reasoning central to CTI analysis and attack technique extraction tasks Cardei [2025].

The incremental generation features of the IFT system (version and timestamp filtering) make it possible to align the training data's recency with AthenaBench's dynamic evaluation horizon, mitigating data leakage while ensuring models are trained on approximately the same vintage of CTI as they are evaluated on Cardei [2025], Alam et al. [2025].

This alignment positions the Athena IFT generator as a natural training counterpart to AthenaBench's evaluation pipeline, much as SecKnowledge and CyberPal models are paired with CTI benchmarks in IBM's work Levi et al. [2024, 2025].

Glaukopis: Template-Based SFT Model for Athena-CTI-LLM

Glaukopis is the template-based supervised fine-tuning (SFT) pipeline used to produce the Athena-CTI-LLM family of cyber security expert small language models. It is the concrete instantiation of the design principles surveyed in earlier sections: a CTI knowledge graph drives a template generator (Section 4); the generator emits IFT triples that are sharded into purpose-built training datasets; those datasets feed a multi-stage chained SFT recipe (Section 1.1); and the resulting checkpoints are evaluated against a portfolio of cyber security benchmarks (Section 5). This section describes each of these elements and how they fit together.

Template-Based IFT as the Training-Data Substrate

Compared to purely LLM-generated instruction datasets, the template-driven, knowledge-grounded approach underlying Glaukopis offers several scientific advantages that motivate its use as the substrate for all SFT stages:

- **Ground-truth faithfulness.** Answers are computed directly from an authoritative CTI graph, reducing hallucinations and inconsistencies in training labels relative to instruction sets generated by an LLM acting as both author and oracle.
- **Explicit coverage control.** Templates can be designed to cover specific combinations of entities (e.g., CAPEC patterns affecting memory resources and their detection methods) or relationships (e.g., malware using two techniques on Linux), and per-template coverage fractions and count limits control dataset size and diversity Cardei [2025].
- **Rich negative examples.** Inequality constraints and relationship filters enable systematic generation of negative examples such as mitigations that do not address a technique or data components that do not detect any technique used by a malware family, exercising the model’s ability to discriminate rather than simply complete.
- **Explainability of templates.** Each template corresponds to a human-readable CTI reasoning pattern, making it easier for security experts to audit and refine the dataset.
- **Incremental refresh.** Version and date filters allow periodic regeneration of updated IFT corpora as CTI sources evolve, paralleling AthenaBench’s dynamic evaluation but on the training side Cardei [2025], Alam et al. [2025].

These properties combine to support a training loop in which new CTI sources or schema updates lead to a refreshed CTI graph; CTI experts design or revise templates for new patterns and tasks; the generator produces updated IFT triples for fine-tuning SLMs; models are evaluated on AthenaBench and complementary suites; and discrepancies and weaknesses inform template revisions and possibly benchmark extensions.

Comparison with IBM’s CyberPal and Other Instruction Pipelines

The Athena approach is conceptually close to SecKnowledge/CyberPal, but with a different emphasis:

- **CyberPal / SecKnowledge.** Uses expert-defined schemas and a pipeline combining LLMs and scripts to synthesize instructions from CTI sources and detection artifacts (e.g., Sigma rules), emphasizing human-curated “expert instruction patterns” and LLM-assisted generation, including CoT answers in SecKnowledge 2.0 Levi et al. [2024, 2025].
- **Athena IFT generator.** Uses a symbolic template language compiled into CTI graph queries, producing triples that are guaranteed to be consistent with the knowledge graph, with strong use of cross-schema constraints and negative examples, and built around incremental, configuration-driven generation Cardei [2025].

The two approaches are complementary. CyberPal demonstrates the effectiveness of expert instruction schemas and CoT enrichment in achieving high cyber security performance, especially in SLMs.

Athena’s system provides precise and reproducible control over content generation aligned with CTI schemas and knowledge-graph structure, potentially feeding into similar CoT-enriched pipelines (e.g., by later adding CoT rationales using LLMs on top of graph-derived answers).

The v21 Template-Generation Vintage

The Glaukopis SFT chain consumes training data produced by a versioned template-generation vintage. The active vintage at the time of writing is **v21** (template directory `tmpl_gen/templates/05182026/`), which is the configuration used for all hyperparameters, dataset structure, and results discussed in the rest of this section. This subsection describes the v21 template-generation flow—how Ariadne (Section 4.3) is projected through three template manifests into the seven dataset shards consumed by the four-stage SFT chain (Section 6.4)—and the reproducibility and lineage discipline that produced it.

Figure 1 summarizes the end-to-end pipeline. Ariadne feeds three template manifests: a *Core* manifest covering Stages 1 and 4, a *TAA* manifest covering Stage 2, and a *CSE* manifest covering Stage 3. Each manifest is processed by the same three-step `make_dataset.sh` pipeline—`docx2json` extracts templates from their authoring format, `tmpl2triples` drives the IFT-generation engine against the Neo4j graph, and `triples2alpaca` merges per-template triple JSON files into a single Alpaca-format dataset (with `instruction`, `input`, and `output` keys mapping to system, prompt, and response respectively). The result is four production training shards plus three matched validation shards, all template-baked and therefore architecture-independent: a single set of seven JSON files feeds every base-model port of the v21 recipe.

Reproducibility provenance.

v21 is a strict reproducibility experiment on top of a predecessor vintage v18.1. The three Sophia-CTI template manifests, the row-count gate JSONs, and the per-stage count limits are `shasum`-verified byte-identical to their v18.1 sources; only dataset names, build-directory names, and Hugging Face push targets change between the two vintages. The v21 sign-off gate was that the post-Core checkpoint would land within ± 1.5 percentage points of the v18.1 Core peak on every benchmark axis. The Total Score landed inside that band, but two per-axis sub-scores (ATE and RCM) drifted outside it, with the same shape that the intermediate v19 and v20 vintages had previously shown against v18.1. The most parsimonious explanation, given that the recipe-side inputs are byte-identical, is data-build non-determinism in the substrate sampler, dedup tiebreaks, and shuffle seeding inside `make_dataset.sh`—a hypothesis confirmed by the fact that pinning the Neo4j read order and seeding the substrate sampler is the next-vintage research direction. The lineage discipline is in our judgement itself a contribution: by carrying byte-identical templates and gates forward across vintages, the Glaukopis pipeline isolates recipe drift from data-layer drift as separable failure modes, and provides a falsification matrix against which future vintages can be evaluated.

The off-plan Recalibrate stage.

The v21 plan as originally specified was a three-stage chain (Core $\rightarrow$ +TAA $\rightarrow$ CSE). A recurring pattern observed across v18.1, v19, v20, and v21 chains is that Stage 3’s narrow CyberSOCEval-letter-set drill reliably trades approximately ten percentage points of VSP

Template generation (tmp1_gen/, v21 / 05182026 vintage)

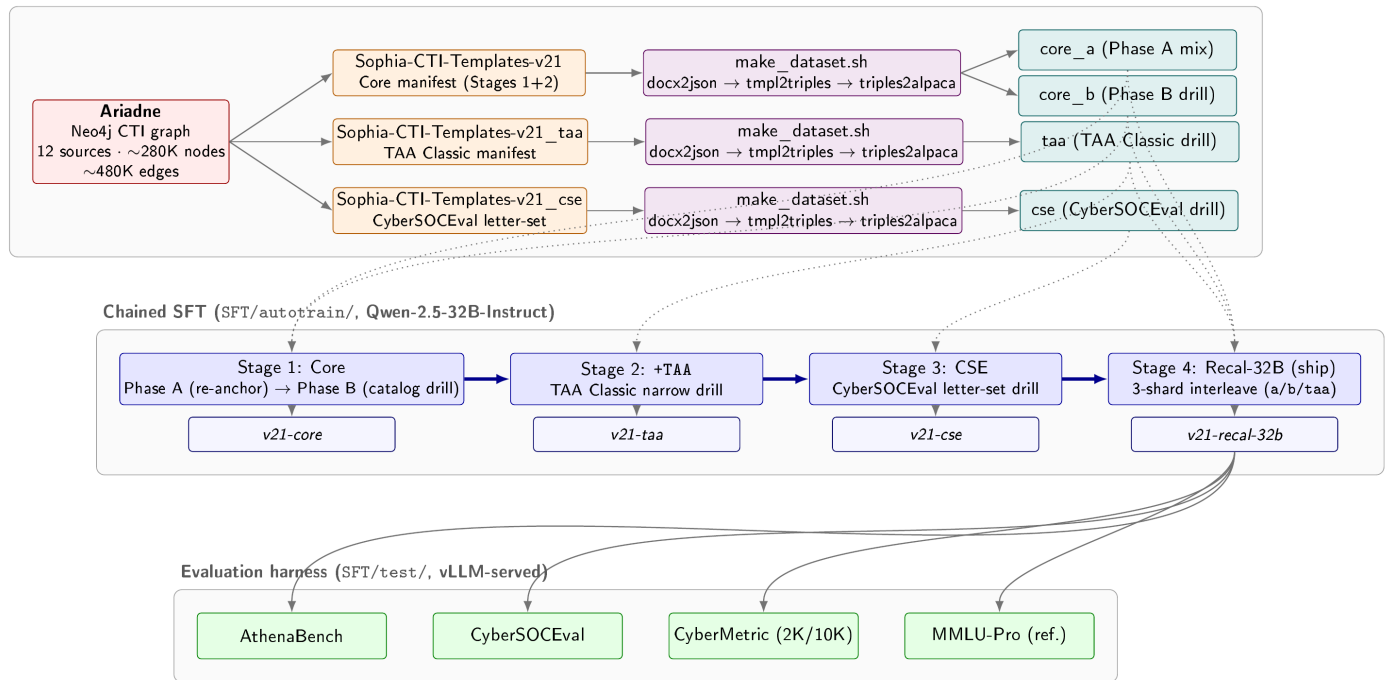


Figure 1 Glaukopis v21 end-to-end flow. Ariadne (Section 4.3), the Athena Neo4j CTI substrate, feeds three v21 template manifests—Core, TAA, and CSE—each processed by the `make_dataset.sh` pipeline into Alpaca-format dataset shards. Four production shards (**core_a**, **core_b**, **taa**, **cse**; matched validation shards omitted for clarity) drive the four-stage chained SFT recipe targeting Qwen-2.5-32B-Instruct, producing the cumulative **v21-core**, **v21-taa**, **v21-cse**, and **ship v21-recal-32b** checkpoints. Dotted arrows mark which shards feed which stages; Stage 4 re-uses Core-A, Core-B, and TAA as a three-shard interleaved touch-up. Each pushed checkpoint is evaluated against the AthenaBench, CyberSOCEval, CyberMetric, and MMLU-Pro suites under a vLLM-served harness.

for the CyberSOCEval-Malware lift it is designed to produce. To recover the VSP regression without sacrificing the Stage 3 gains, an off-plan fourth stage—*Recalibrate*—was added as a three-shard interleaved touch-up over Core-A, Core-B, and TAA. At the 14B base scale, a single Recalibrate recipe (1e-6 LR, 0.25/0.40/0.35 interleave probabilities, `--max-samples 2400`) cleanly recovers VSP. At the 32B base scale, the same recipe sits at the `adamw_8bit` optimizer noise floor and drifts VSP the wrong way; the recipe was therefore re-tuned (3e-6 LR, 0.15/0.60/0.25 Phase-B-heavy mix, `--max-samples 3600`) to expose more VSP/RMS catalog per interleaved row while holding step count and wall-time constant. The 32B Stage 4 split is the basis of the ship checkpoint discussed in Section 6.4.

Architecture ports.

Stages 1 through 3 carry the same recipe byte-identically across all five base-model ports of v21: Qwen-2.5-32B-Instruct (canonical), Qwen-2.5-14B-Instruct, Foundation-Sec-8B, Llama-3.1-8B-Instruct, and Qwen3-30B-A3B-Thinking-2507 (a thinking-mode MoE). The fact that the same template-baked shards drive all ports is what makes cross-architecture comparison meaningful: any axis-level differences between checkpoints derived from different bases under the same recipe and same data are attributable to base-model differences rather than training-data drift. The Qwen3-MoE port is also the first port to exercise a thinking-on / thinking-off A/B at serve time, exposing the SFT contribution to no-trace inference behavior separately from the base model’s reasoning gain. Stage 4 is the only stage where the recipe does not carry identically across architectures:

at the 32B dense scale the Phase-B-heavy 3e-6 LR variant is required (as discussed above), and on the Qwen3-MoE port Stage 4 closed without a beat against **v21-cse** on either Stage-4 variant, so the on-chain ship for that port is **v21-cse**.

The Glaukopis SFT Chain

The Glaukopis SFT pipeline is a four-stage chain over template-generated CTI instruction data. The canonical configuration targets Qwen-2.5-32B-Instruct as the base model and produces a sequence of four cumulative checkpoints—Core, +TAA, CSE, and Recalibrated-32B—of which the last is the production ship checkpoint. The same recipe ports byte-identically to four other base architectures (Qwen-2.5-14B-Instruct, Foundation-Sec-8B, Llama-3.1-8B-Instruct, and the Qwen3-30B-A3B-Thinking-2507 MoE), with only the base-model pointer and per-architecture memory flags changing between ports. The four-stage structure is itself the load-bearing element of the recipe; the per-stage details are described below.

Dataset shards.

The chain consumes seven dataset shards produced by the template generator. The **Core-A** shard is a broad re-anchor mix combining knowledge-base, multiple-choice, threat-actor-attribution (TAA), SOC-style, CyberMetric-style, mitigation-strategy, and yes/no instruction families. The **Core-B** shard is an AthenaBench catalog drill targeting Risk Mitigation Strategy (RMS), Attack Technique Extraction (ATE), Vulnerability Severity Prediction (VSP), and

Root Cause Mapping (RCM) tasks Alam et al. [2025]. The **TAA** shard is a narrow drill on TAA Classic. The **CSE** shard is a CyberSOCEval letter-set drill, structured to exercise the format and reasoning pattern used by the CyberSOCEval malware and threat-intel test suites Meta AI / Purple Llama Team [2025]. Three matched validation shards (**Core-Val**, **TAA-Val**, **CSE-Val**) hold out a per-axis slice (50 rows per shortname) for held-out evaluation during training. All shards are template-baked and therefore architecture-independent: a single set of seven JSON files feeds every base-model port.

Stage 1: Core (broad re-anchor + AthenaBench catalog drill).

Stage 1 runs as two phases inside a single launcher, with only the final phase’s merged checkpoint pushed. *Phase A* is the broad re-anchor: it trains on the Core-A shard together with a general-purpose instruction mix (the Tulu-3 SFT mixture and an Alpaca-English demonstration set) at sequence length 8192, packing on, learning rate 1×10^{-5} , and effective batch size 16. The intent is to re-establish broad instruction-following behavior on top of a CTI vocabulary base. *Phase B* drills on the Core-B AthenaBench-catalog shard at sequence length 16384 with packing off (so that long structured outputs are not truncated by packing boundaries), learning rate 5×10^{-6} , and effective batch size 8. The shorter, lower-LR Phase B is calibrated to imprint precise AthenaBench task structure on top of the broad foundation laid by Phase A, rather than to continue broad assimilation Alam et al. [2025]. Together, Phase A and Phase B implement at fine granularity the principle from FLAN-T5 and “Don’t Stop Pretraining” that broad domain exposure and narrow task supervision benefit from separate optimization regimes Chung et al. [2022], Gururangan et al. [2020].

Stage 2: +TAA narrow drill.

Stage 2 takes the Core checkpoint as its base and trains on the TAA shard alone, at sequence length 4096, packing on, learning rate 5×10^{-6} , and effective batch size 16. The objective is to harden the model’s TAA Classic performance without disturbing the broader catalog skills established in Stage 1.

Stage 3: CSE letter-set drill.

Stage 3 takes the +TAA checkpoint and trains on the CSE shard at the same sequence length, packing, learning rate, and effective batch size as Stage 2. This stage is targeted preparation for the CyberSOCEval malware and threat-intelligence suites Meta AI / Purple Llama Team [2025], and is the point at which a known trade-off appears: lifting the CyberSOCEval axis through this drill comes at a measured cost to VSP performance from Stage 1’s Phase B, which the next stage exists to recover.

Stage 4: Recalibrated touch-up (ship variant).

Stage 4 is a three-shard interleaved touch-up trained off the Stage 3 (CSE) checkpoint with the explicit goal of recovering the VSP regression introduced by Stage 3 without losing Stage 3’s CyberSOCEval gains. The ship variant trains on a Core-A / Core-B / TAA mix at relative sampling probabilities 0.15/0.60/0.25 (i.e., Phase-B-heavy), using LlamaFactory’s `interleave_under` mixing strategy with a 3600-row sample cap, at sequence length 16384, packing off, learning rate 3×10^{-6} , and effective batch size 8. The 14B-tuned variant of this recipe (lower learning rate, more balanced

mixture, smaller sample cap) is retained as a recipe-provenance A/B against the ship-grade 32B recipe, but in the 32B-target chain it is the higher-LR, Phase-B-heavy variant that recovers VSP and tops the v21 evaluation leaderboard.

Cross-cutting training-system details.

All four stages share the same systems-level configuration: bf16 mixed precision, DeepSpeed ZeRO-3 (with optional CPU offload), the `adamw_8bit` optimizer (mandatory at the 32B scale to fit activations), Liger kernels for fused attention paths, and gradient checkpointing on at 32B regardless of GPU count. The 8×H100-80GB SXM topology is the canonical training target; 8×H200-141GB SXM and 8×RTX-PRO-6000-96GB are supported alternates. Effective batch size is held constant across launch topologies by auto-scaling gradient accumulation to GPU count.

Why four stages rather than one.

The four-stage structure embodies the chained-SFT rationale from Section 1.1 in a CTI-specific form. Stages 1A and 1B separate broad domain absorption from precise structured-task imprinting, each at its own learning rate and packing regime Gururangan et al. [2020], Chung et al. [2022]. Stages 2 and 3 carry out narrow drills against specific benchmark axes (TAA and CyberSOCEval respectively) without re-perturbing the broader skill set learned in Stage 1. Stage 4 is a recalibration step that recovers regressed axes (here, VSP) through controlled three-shard interleaving and a slightly higher learning rate, exploiting the fact that all template-generated shards remain available as training corpora throughout the chain. The result is a sequence of intermediate checkpoints—Core, +TAA, CSE, Recalibrated-32B—that can each be evaluated independently against the benchmark portfolio of Section 5, supporting controlled per-stage ablation in addition to producing a single ship checkpoint. This per-stage observability is itself a benefit of the chained structure: any axis-level regression observed at evaluation time is attributable to a specific stage and a specific dataset shard, and can be addressed by re-running only the affected stages rather than the full pipeline.

Evaluation Harness

Each pushed checkpoint in the Glaukopis chain is evaluated against the portfolio described in Section 5, consisting of **AthenaBench** (the six CTI tasks: CKT/MCQ, RCM, VSP, ATE, TAA Classic and Canonical, and RMS) Alam et al. [2025]; **CyberSOCEval** (the malware and threat-intelligence test suites) Meta AI / Purple Llama Team [2025]; **CyberMetric** at the 2000- and 10000-question tiers Tihanyi et al. [2024]; and **MMLU-Pro** as a decoupled general-intelligence reference. Serving is via vLLM in an OpenAI-compatible configuration; the same alias-keyed harness scores public baselines through hosted inference providers for direct comparison. Per-task token and wall-clock usage is captured so that benchmark sweeps are accompanied by a cost summary covering both API and local-GPU compute, which lets evaluation cost be reported alongside accuracy as a first-class metric.

The mapping between training stages and evaluation suites is deliberate: AthenaBench RMS/ATE/VSP/RCM are exercised by Stage 1 Phase B’s Core-B shard; AthenaBench TAA Classic and TAA Canonical are exercised by Stage 2; CyberSOCEval malware and threat-intel reasoning are exercised by Stage 3; CyberMetric and the AthenaBench MCQ axis are exercised primarily by Stage 1 Phase A’s Core-A shard. The Stage 4 recalibration is designed

not to alter this mapping but to recover any axis that has regressed across the prior stages, with the 32B-tuned, Phase-B-heavy mixture specifically chosen to restore VSP without sacrificing CyberSOCEval performance. MMLU-Pro is run on a separate session so that suite scope stays decoupled from CTI scoring: it is reported as a sanity check on general capability rather than as a target for optimization.

Toward Cyber Security Expert Small Language Models

Bringing together the elements above, an end-to-end cyber security expert SLM pipeline of the kind exemplified by Glaukopis can be summarized as:

1. **CTI graph construction.** Continuously ingest and normalize CTI sources (ATT&CK, CVE, CAPEC, CWE, ENGAGE, ICS/OT sources, proprietary feeds) into a Neo4j or similar graph.
2. **Template authoring.** CTI experts encode important reasoning patterns (knowledge recall, root cause mapping, severity estimation, attribution clues, mitigation reasoning) as templates in the Athena language.
3. **IFT generation.** Run the generation tool with version and date constraints aligned to a training window, producing large, structured IFT datasets per task family Cardei [2025].
4. **Chained SFT.** Fine-tune SLMs (e.g., 8–32B open models) using a chain of stages as described in Section 6.4, with the dataset-shard / stage mapping aligned to the benchmark portfolio.
5. **Evaluation.** Benchmark models on AthenaBench, CyberSOCEval, CyberMetric, and complementary suites (CTIBench, SecEval, CyberBench, SEvenLLM-Bench) to assess generalization and compare with larger proprietary models Alam et al. [2024, 2025], Hu et al. [2024], Liu et al. [2024], Ji et al. [2024], Tihanyi et al. [2024], Meta AI / Purple Llama Team [2025].
6. **Feedback and iteration.** Use benchmark error patterns to design new templates, adjust CTI graph coverage, or refine IFT data (e.g., adding more negative cases or multi-hop chains), and re-run only the affected SFT stages.

This pipeline is closely aligned with IBM’s vision of cyber security expert SLMs Levi et al. [2024, 2025], but anchored in Athena’s graph-driven IFT, the chained-SFT recipe described above, and dynamic CTI benchmarking via AthenaBench and CyberSOCEval.

Open Challenges and Research Directions

Despite rapid progress, several open questions remain:

- **Data leakage and benchmark freshness.** Dynamic benchmarks like AthenaBench reduce, but do not eliminate, the risk that training data (e.g., from the same CTI sources) overlaps heavily with evaluation samples. Careful time-based splits and provenance tracking between the IFT generator and benchmarks will be crucial Alam et al. [2025], Cardei [2025].
- **Balancing symbolic templates and LLM creativity.** Template-based generation offers precision and coverage, but LLM-generated paraphrases can increase linguistic diversity and realism (e.g., multiple phrasings of the same CTI scenario). Hybrid pipelines that use templates to define semantics and LLMs to diversify surface form—while preserving correctness—are a promising area.

- **Incorporating chain-of-thought and explanations.** SecKnowledge 2.0 shows that CoT rationales significantly improve cyber reasoning Levi et al. [2025]. Extending Athena templates to generate structured reasoning steps (via explicit graph traversal reasoning) and then augmenting them with LLM-generated natural language CoT could yield explainable CTI SLMs.
- **Operational evaluation dimensions.** AthenaBench focuses on knowledge and reasoning quality. Security operations also care about robustness to prompt injection, misuse, adversarial CTI, latency, and integration with SIEM/SOAR systems. Benchmarks like SecEval and CyberBench partly address this; extending AthenaBench and complementary suites to cover these aspects remains an important direction Hu et al. [2024], Liu et al. [2024], Alam et al. [2025].
- **Multilingual and cross-regional CTI.** SEvenLLM emphasizes bilingual corpora Ji et al. [2024]. Current CTI benchmarks primarily use English sources, omitting intelligence from other languages that often contain region-specific threats. Template-based IFT over multilingual CTI graphs and multilingual benchmark tasks could help close this gap.
- **Human-in-the-loop and analyst trust.** For high-stakes CTI decisions, analysts need to understand why the model suggested a mapping or mitigation and how it connects to CTI sources. Template-driven data, CTI graph citations, and CoT rationales may enable verifiable, inspectable explanations.

Conclusion

Instruction fine-tuning and small language models are emerging as a practical and powerful combination for cyber security, especially in CTI workflows that demand structured reasoning, up-to-date knowledge, and on-premises deployment. IBM’s CyberPal and SecKnowledge series demonstrate that expert-curated, CTI-grounded instruction datasets can turn SLMs into strong cyber security assistants, while AthenaBench raises the bar for CTI evaluation by integrating live CTI sources and focusing on reasoning-intensive tasks Levi et al. [2024, 2025], Alam et al. [2025].

The Athena IFT Dataset Generation System designed by Cardei and collaborators provides a complementary training-time infrastructure: a template language over a CTI knowledge graph that can generate rich, grounded, and incrementally refreshable instruction triples across multiple CTI schemas and tasks Cardei [2025]. Together, these elements outline a coherent research program:

- Build and maintain high-fidelity CTI graphs from authoritative sources.
- Encode analyst reasoning patterns as templates that produce IFT data.
- Train cyber security expert SLMs using these datasets.
- Evaluate them with dynamic CTI benchmarks like AthenaBench and iteratively refine both data and models.

The result is a principled pathway toward cyber security expert small language models grounded simultaneously in CTI knowledge bases, instruction fine-tuning theory, and rigorous CTI benchmarking.

Conflicts of interest

None declared.

Funding

No external funding supported this manuscript.

Data availability

No datasets were generated or analyzed during the preparation of this survey paper.

Acknowledgments

The authors thank colleagues at Florida Atlantic University's Department of Electrical Engineering and Computer Science and at Athena Security Group for discussions that informed this work.

References

- Common attack pattern enumeration and classification (CAPEC). <https://capec.mitre.org/>, a. Accessed 2025-12-03.
- Common vulnerabilities and exposures (CVE). <https://www.cve.org/>, b. Accessed 2025-12-03.
- Common weakness enumeration (CWE). <https://cwe.mitre.org/>, c. Accessed 2025-12-03.
- M. T. Alam, D. Bhusal, N. Shah, and N. Rastogi. Ctibench: A benchmark for evaluating LLMs in cyber threat intelligence. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. URL <https://arxiv.org/abs/2406.07508>.
- M. T. Alam, D. Bhusal, S. Ahmad, N. Rastogi, and P. J. Worth. Athenabench: A dynamic benchmark for evaluating LLMs in cyber threat intelligence. *arXiv preprint arXiv:2511.01144*, 2025. URL <https://arxiv.org/abs/2511.01144>.
- M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana, S. Frolov, R. P. Giri, D. Kapil, Y. Kozyrakis, D. LeBlanc, J. Milazzo, A. Straumann, G. Synnaeve, V. Vontimitta, S. Whitman, and J. Saxe. Purple Llama CyberSecEval: A secure coding benchmark for language models. *arXiv preprint arXiv:2312.04724*, 2023. URL <https://arxiv.org/abs/2312.04724>.
- M. Bhatt, S. Chennabasappa, Y. Li, C. Nikolaidis, D. Song, S. Wan, F. Ahmad, C. Aschermann, Y. Chen, D. Kapil, D. Molnar, S. Whitman, and J. Saxe. CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models. *arXiv preprint arXiv:2404.13161*, 2024. URL <https://arxiv.org/abs/2404.13161>.
- I. Cardei. Instruction fine tuning dataset design. Technical report, Athena Labs, Athena Security Group, 2025. Internal design document describing the Athena CTI Neo4j graph schema and template-based IFT dataset generation.
- H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022. URL <https://arxiv.org/abs/2210.11416>.
- C. Fieblinger, A. M. Tjoa, et al. Actionable cyber threat intelligence using knowledge graphs and large language models. *arXiv preprint*, 2024.
- S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8342–8360, 2020. URL <https://arxiv.org/abs/2004.10964>.
- H. Hu, P. Zhang, et al. Seceval: A comprehensive benchmark for evaluating cybersecurity knowledge in foundation models. *arXiv preprint*, 2024.
- H. Ji, J. Yang, L. Chai, C. Wei, L. Yang, Y. Duan, Y. Wang, T. Sun, H. Guo, T. Li, et al. Sevenllm: Benchmarking, eliciting, and enhancing abilities of large language models in cyber threat intelligence. *arXiv preprint arXiv:2405.03446*, 2024. URL <https://arxiv.org/abs/2405.03446>.
- M. Levi, Y. Allouche, D. Ohayon, and A. Puzanov. Cyberpal.ai: Empowering LLMs with expert-driven cybersecurity instructions. *arXiv preprint arXiv:2408.09304*, 2024. URL <https://arxiv.org/abs/2408.09304>.
- M. Levi, D. Ohayon, A. Blobstein, R. Sagi, I. Molloy, and Y. Allouche. Toward cybersecurity-expert small language models. *arXiv preprint arXiv:2510.14113*, 2025. URL <https://arxiv.org/abs/2510.14113>.
- N. Li, A. Pan, A. Gopal, S. Yue, D. Berrios, A. Gatti, J. D. Li, A.-K. Dombrowski, S. Goel, L. Phan, G. Mukobi, N. Helm-Burger, R. Lababidi, L. Justen, A. B. Liu, M. Chen, I. Barrass, O. Zhang, X. Zhu, R. Tamirisa, B. Bharathi, A. Khoja, Z. Zhao, A. Herbert-Voss, C. B. Breuer, S. Marks, O. Patel, A. Zou, M. Mazeika, Z. Wang, P. Oswal, W. Lin, A. A. Hunt, J. Tienken-Harder, K. Y. Shih, K. Talley, J. Guan, R. Kaplan, I. Steneker, D. Campbell, B. Jokubaitis, A. Levinson, J. Wang, W. Qian, K. K. Karmakar, S. Basart, S. Fitz, M. Levine, P. Kumaraguru, U. Tupakula, V. Varadharajan, R. Wang, Y. Shoshitaishvili, J. Ba, K. M. Esvelt, A. Wang, and D. Hendrycks. The WMDP benchmark: Measuring and reducing malicious use with unlearning. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL <https://arxiv.org/abs/2403.03218>.
- Z. Liu, J. Shi, and J. F. Buford. Cyberbench: A multi-task benchmark for evaluating large language models in cybersecurity. *arXiv preprint arXiv:2407.08665*, 2024. URL <https://arxiv.org/abs/2407.08665>.
- R. McMillan. Defining threat intelligence. Technical report, Gartner, 2013.
- Meta AI / Purple Llama Team. CyberSOCEval: Benchmarking LLMs for malware analysis and threat intelligence reasoning. <https://github.com/meta-llama/PurpleLlama>, 2025. *arXiv preprint; exact citation form to be confirmed*.
- L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022. URL <https://arxiv.org/abs/2203.02155>.
- B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas. MITRE ATT&CK: Design and philosophy. Technical Report MP180360, The MITRE Corporation, 2018. URL <https://attack.mitre.org/resources/>.

- N. Tihanyi, M. A. Ferrag, R. Jain, T. Bisztray, and M. Debbah. CyberMetric: A benchmark dataset based on retrieval-augmented generation for evaluating LLMs in cybersecurity knowledge. *arXiv preprint arXiv:2402.07688*, 2024. URL <https://arxiv.org/abs/2402.07688>.
- J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022a. URL <https://arxiv.org/abs/2109.01652>.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022b. URL <https://arxiv.org/abs/2201.11903>.
- X. Wu, L. Wang, et al. Open-cykg: Constructing a cyber threat intelligence knowledge graph from open sources. *arXiv preprint*, 2022.
- Y. Yu, J. Kang, et al. Ctikg: A large-scale, high-quality cyber threat intelligence knowledge graph. *arXiv preprint*, 2024.
- Y. Zhang, H. Chen, et al. Cyberkg: A cyber threat intelligence knowledge graph for threat analysis. *arXiv preprint*, 2023.
- Z. Zhao, H. Li, et al. K-ctiaa: Knowledge graph enhanced cyber threat intelligence action analysis. *arXiv preprint*, 2024.